Personalized Programming & Backend Engineering Course

A 12-week intensive roadmap + EV capstone project

Designed around an existing Python/Django backend foundation, with the goal of expanding into Go, TypeScript, Rust, systems, databases, Docker, testing, CI/CD and system design.

Primary languages	Go → TypeScript → Rust
Supporting skills	PostgreSQL, Linux, Docker, Git, testing, CI/CD, networking, system design, DSA
Course duration	12 weeks + 3–5 weeks for the EV capstone
Recommended daily time	4–6 hours/day

Core principle: don't just watch tutorials. For every hour of video/documentation, spend roughly two hours writing code.

1. Recommended Direction

You already have practical experience with Python/Django, APIs, databases, Git, Linux/server-side debugging and Docker. The goal is therefore not to relearn programming from zero. Instead, the course deliberately puts you into ecosystems that force different ways of thinking.

Primary languages

Go — first priority	Backend engineering, concurrency, networking, microservices, CLI tools, performance, systems thinking
TypeScript — second priority	Strongly typed JavaScript, Node.js backend, modern frontend, full-stack development, ecosystem exposure
Rust — third priority	Memory management, ownership/borrowing, low-level thinking, performance, systems programming

Alongside them

- PostgreSQL / SQL
- Linux
- Networking
- Docker
- Testing
- Git
- CI/CD
- System design
- Data structures & algorithms

The intended outcome is a much stronger engineering profile than simply knowing three programming languages.

2. Daily Learning Structure

Recommended daily workload: 4–6 hours. Scale this down if necessary, but keep the same proportions.

Time	Activity
1.5 hr	Learn — documentation, concepts, selected videos
2 hr	Code — exercises and implementation
1 hr	Project — apply what you learned
30 min	DSA — one or more focused problems
30 min	Read / debug / review

Golden rule

> **Don't watch a 3-hour tutorial and call that learning. For every 1 hour of video/documentation, spend roughly 2 hours writing code.**

3. Phase 0 — Week 1: Become Dangerous With Your Development Tools

Before learning Go, become much better at the environment around your code.

Learn

- Linux shell
- Processes
- Pipes and redirects
- grep / sed / awk
- curl
- HTTP
- Ports
- DNS
- Environment variables
- Git internals
- Debugging
- Profiling
- Package management
- CI/CD basics

Recommended resource

MIT Missing Semester 2026 — covers shell, development tools, debugging/profiling, Git, packaging/shipping code and code quality.

Exercises

```
find .
grep
awk
sed
curl
netstat / ss
ps
top
lsof
```

Then write shell scripts. Example target:

```
./find_errors.sh application.log

Total errors: 172
```

500 errors: 31
404 errors: 98
timeout errors: 43
other: 0

Goal by the end of Week 1

You should be comfortable debugging a server without immediately opening your IDE.

4. Phase 1 — Weeks 2–5: Go

Go is the first major language because it exposes concepts that Python/Django often abstracts away.

You will encounter

- goroutines
- channels
- interfaces
- pointers
- structs
- error handling
- memory
- concurrency
- networking
- compilation
- static typing

Official and video resources

Go Learn — official learning material, including the Tour of Go.

Tour of Go

Go by Example

Go Programming — freeCodeCamp — a video companion that eventually builds an RSS aggregator using HTTP, PostgreSQL, authentication and workers.

Week 2 — Go Fundamentals

Learn:

- variables
- constants
- functions
- if/else
- loops
- arrays

- slices
- maps
- structs
- methods
- pointers
- packages
- modules

Exercises

- calculator
- word counter
- CSV parser
- JSON parser
- URL shortener
- log parser

Don't use frameworks for these exercises.

Week 3 — Go's Interesting Stuff

Interfaces

```
type Animal interface {
    Speak() string
}
```

Understand how Go interfaces differ significantly from Python inheritance/ABC patterns.

Error handling

```
if err != nil {
    return err
}
```

Then learn:

- errors.Is
- errors.As
- fmt.Errorf
- custom errors
- wrapping

Generics

```
func Map[T any](items []T, fn func(T) T) []T
```

Testing

- testing package
- table-driven tests
- benchmarks

Week 4 — Concurrency

Learn:

- goroutines
- channels
- select
- mutexes
- WaitGroup
- context
- worker pools
- race conditions
- deadlocks

Build: Concurrent URL Checker

Input:

```
urls.txt
```

Your program should use 10 workers, a URL queue and HTTP requests, then produce results such as:

```
google.com      200   142ms
github.com      200   210ms
example.com     404    82ms
foo.com         TIMEOUT
```

Then intentionally introduce a race condition and detect it.

Week 5 — Go Backend

Learn:

- HTTP
- REST
- middleware
- routing
- JSON
- authentication
- PostgreSQL
- transactions
- connection pooling
- logging
- configuration

Build a User API

```
POST   /users
GET    /users
GET    /users/:id
```

```
PUT    /users/:id
DELETE /users/:id
```

Then add authentication.

Important learning outcome

Go should make you understand what Django was doing for you. That is exactly the point.

5. Phase 2 — Weeks 6–8: TypeScript + Node.js

TypeScript gives you exposure to the enormous JavaScript ecosystem while providing static typing.

Resources

TypeScript Handbook — language, type system, generics, modules and advanced type manipulation.

TypeScript Crash Course — freeCodeCamp

Full Stack Open — University of Helsinki — React, Node.js, MongoDB, GraphQL, TypeScript, testing, databases, containers and CI.

Week 6 — JavaScript → TypeScript

First learn enough JavaScript to understand:

- let / const
- objects
- arrays
- functions
- closures
- callbacks
- promises
- async/await
- modules
- event loop

Then learn TypeScript:

- types
- interfaces
- type aliases
- union types
- intersection types
- generics
- enums
- narrowing

- utility types

Especially understand:

```
string | null
```

versus:

```
any
```

Understand why any can destroy much of TypeScript's value.

Week 7 — Node.js

Learn:

- Node runtime
- npm
- package.json
- modules
- async I/O
- event loop
- Express/Fastify
- REST APIs
- middleware
- authentication
- validation
- testing

Build:

```
TypeScript API
      ↓
PostgreSQL
      ↓
Redis
```

Week 8 — Full Stack

Learn enough React to understand:

- components
- props
- state
- hooks
- forms
- API calls
- routing
- authentication

You do not need to become a frontend specialist.

6. Phase 3 — Weeks 9–10: Rust

Rust is intentionally harder. It gives you a completely different mental model and deeper understanding of memory and resource management.

Start with The Rust Programming Language

Practice with Rustlings

The official Rust learning page recommends the Rust Book, Rust by Example and Rustlings as complementary learning paths.

Week 9 — Core Rust

Learn:

- variables
- mutability
- ownership
- borrowing
- references
- structs
- enums
- pattern matching
- Option
- Result
- modules
- Cargo

Most important concepts:

- Ownership
- Borrowing
- Lifetimes

Do not rush these concepts. Genuine understanding of ownership should improve your overall understanding of memory and resource management.

Week 10 — Rust Deeper Dive

Learn:

- traits
- generics
- iterators
- closures
- smart pointers
- threads

- channels
- async
- Tokio
- error handling

Build: Rust CLI

```
rgrep "ERROR" ./logs/
```

It should recursively search files concurrently. Then add:

- regex
- colored output
- parallel processing
- JSON output
- configuration

7. Phase 4 — Week 11: Databases + Docker + Systems

PostgreSQL

Upgrade your database understanding. Study:

- indexes
- B-tree
- transactions
- ACID
- isolation levels
- locking
- deadlocks
- EXPLAIN
- query planning
- CTEs
- window functions
- partitioning
- connection pooling

PostgreSQL Tutorial — relational concepts, SQL, joins, aggregates, transactions and window functions.

PostgreSQL Documentation

Exercise: take a slow query, run EXPLAIN ANALYZE, optimize it, and be able to explain why it became faster.

```
EXPLAIN ANALYZE
SELECT ...;
```

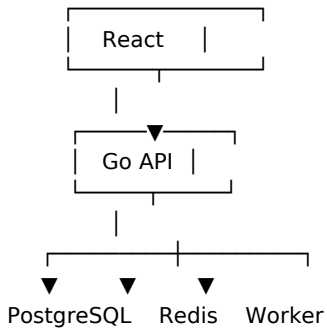
Docker

Learn:

- images
- containers
- layers
- Dockerfile
- volumes
- networks
- Compose
- health checks
- multi-stage builds
- container security
- resource limits
- registries

Docker Get Started — containers, images, application stacks and Compose.

Target stack:



Run the entire stack through Docker Compose.

8. Phase 5 — Week 12: System Design + Interview Preparation

Now stop learning languages and start learning engineering.

Study:

- HTTP
- DNS
- TCP
- load balancers
- caching
- queues
- pub/sub

- databases
- replication
- sharding
- rate limiting
- authentication
- authorization
- observability
- horizontal scaling
- event-driven architecture

DSA

Target roughly 3–5 problems/day, focusing on patterns rather than chasing 500 problems.

- arrays
- hash maps
- two pointers
- sliding window
- binary search
- stack
- queue
- linked list
- trees
- graphs
- heap
- backtracking
- dynamic programming

9. 12-Week Roadmap at a Glance

Week	Focus	Main outcome
1	Linux / Git / debugging	Strong developer tooling
2	Go	Language fundamentals
3	Go	Interfaces / errors / testing
4	Go	Concurrency
5	Go	Production API
6	TypeScript	Type system
7	Node.js	Backend

8	React	Full stack
9	Rust	Ownership / memory
10	Rust	Concurrency / CLI
11	PostgreSQL + Docker	Infrastructure
12	System Design	Architecture
13+	EV	Capstone

Do not necessarily stop at Week 12. Treat the 12 weeks as the boot camp, then spend another 3–5 weeks building EV.

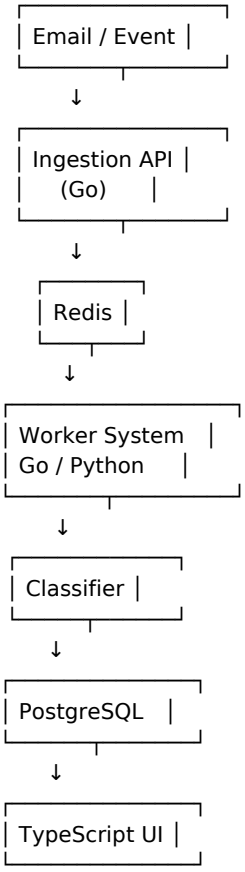
10. Final Capstone — EV: Developer Intelligence Platform

Do not build another TODO application. Build a system that combines the concepts from the entire course.

Concept

EV ingests emails/events/logs and intelligently classifies and processes them.

Architecture



11. EV Requirements

1. Ingestion

Create:

```
POST /events
```

Accept:

```
{
  "source": "email",
  "sender": "someone@example.com",
  "subject": "Your payment is due",
  "body": "Your EMI is due tomorrow..."
}
```

2. Classification

Classify events into:

- IMPORTANT
- FINANCE
- WORK
- SOCIAL
- PROMOTIONAL
- SPAM
- OTHER

Initially use rules. Then optionally introduce an ML/LLM classifier.

3. Async Processing

Do not classify synchronously. Use:

```
API
 ↓
Redis queue
 ↓
Worker
 ↓
Classifier
 ↓
PostgreSQL
```

This forces you to understand asynchronous architecture.

4. Go Service

The ingestion service must be written in Go.

Requirements:

- REST API
- middleware
- authentication
- validation
- PostgreSQL
- Redis
- structured logging
- tests
- graceful shutdown
- configuration

5. TypeScript Dashboard

Build a dashboard showing:

- Total emails

- Important
- Finance
- Work
- Spam
- Processing
- Failed

Also build a searchable inbox.

6. Rust Component

Use Rust for a component where its characteristics are useful. Build a small Rust service for large log parsing.

```
10 GB application log
        ↓
       Rust
        ↓
extract:
    errors
    latency
    IP
    endpoint
    status
```

You do not need Rust everywhere. The lesson is to use different languages where their characteristics actually make sense.

7. Dockerize Everything

```
ev/
│
├── services/
│   ├── ingestion-go/
│   ├── worker-go/
│   └── log-parser-rust/
│
├── frontend/
│   └── typescript-react/
│
├── infrastructure/
│   ├── docker/
│   └── postgres/
│
├── docker-compose.yml
│
├── docs/
│   ├── architecture.md
│   └── decisions.md
│
└── README.md
```

12. Final Knowledge Test

Once EV is complete, stop looking at tutorials. Use these challenges to test whether you understand the system rather than merely knowing how to assemble it.

Challenge 1 — High-traffic 500s

Your API suddenly starts returning 500 only when traffic exceeds 100 requests/sec. Find the problem.

Challenge 2 — Duplicate processing

Two workers process the same email. Prevent duplicate processing.

Challenge 3 — PostgreSQL CPU

PostgreSQL CPU reaches 100%. Find the slow query.

Challenge 4 — Redis failure

Redis goes down. Your application must degrade gracefully.

Challenge 5 — Worker crash

One worker crashes halfway through processing. What happens to the job? Design the behavior.

Challenge 6 — Concurrency

Your Go API receives 1000 concurrent requests. How does your application handle them?

Challenge 7 — Python vs Rust

The Rust log parser is 5× faster than your Python implementation. Explain why.

Challenge 8 — Container failure

Your Docker container works locally but fails in production. Debug it.

Challenge 9 — Scale EV

Design EV to handle 10 million events/day without simply throwing more servers at it.

13. What You Should Know After 12 Weeks

You should be able to look at an architecture such as:

```
Client
  ↓
Load Balancer
  ↓
Go API
  ↓
Redis
  ↓
Workers
  ↓
PostgreSQL
```

and explain:

- why every component exists
- what happens when each component fails
- how the system scales

- where the bottlenecks are

That is much more valuable than simply being able to say: “I know Go, Rust and TypeScript.”

14. How to Make the Capstone Interview-Ready

Because this project should become evidence of engineering ability, not just a tutorial exercise, make it GitHub-quality.

- Write a clear README.
- Include an architecture diagram.
- Document important architecture decisions.
- Write meaningful unit/integration tests.
- Use Docker Compose for local setup.
- Add CI.
- Document the API.
- Benchmark relevant components.
- Document failure scenarios and recovery behavior.
- Keep meaningful Git commits.
- Record a short demo video.

The final project should give you concrete material to discuss in interviews instead of simply listing new languages on your résumé.

15. Recommended Learning Philosophy

1. Learn a concept.
2. Implement it without a framework where practical.
3. Break it intentionally.
4. Debug it.
5. Write tests for it.
6. Use it inside EV.
7. Explain why it works.
8. Explain how it fails.
9. Explain how it scales.

If you follow this sequence consistently, you won't just add three languages to your toolkit—you'll deepen your understanding of backend and systems engineering.

16. Resource Index

Topic	Resource
Linux / developer tools	MIT Missing Semester 2026
Go	Go Learn
Go interactive	Tour of Go
Go examples	Go by Example
Go video	freeCodeCamp Go course
TypeScript	TypeScript Handbook
TypeScript video	freeCodeCamp TypeScript course
Full stack	Full Stack Open
Rust	The Rust Programming Language
Rust practice	Rustlings
Rust learning	Rust Learn
PostgreSQL	PostgreSQL Tutorial
PostgreSQL docs	PostgreSQL Documentation
Docker	Docker Get Started

End of course document